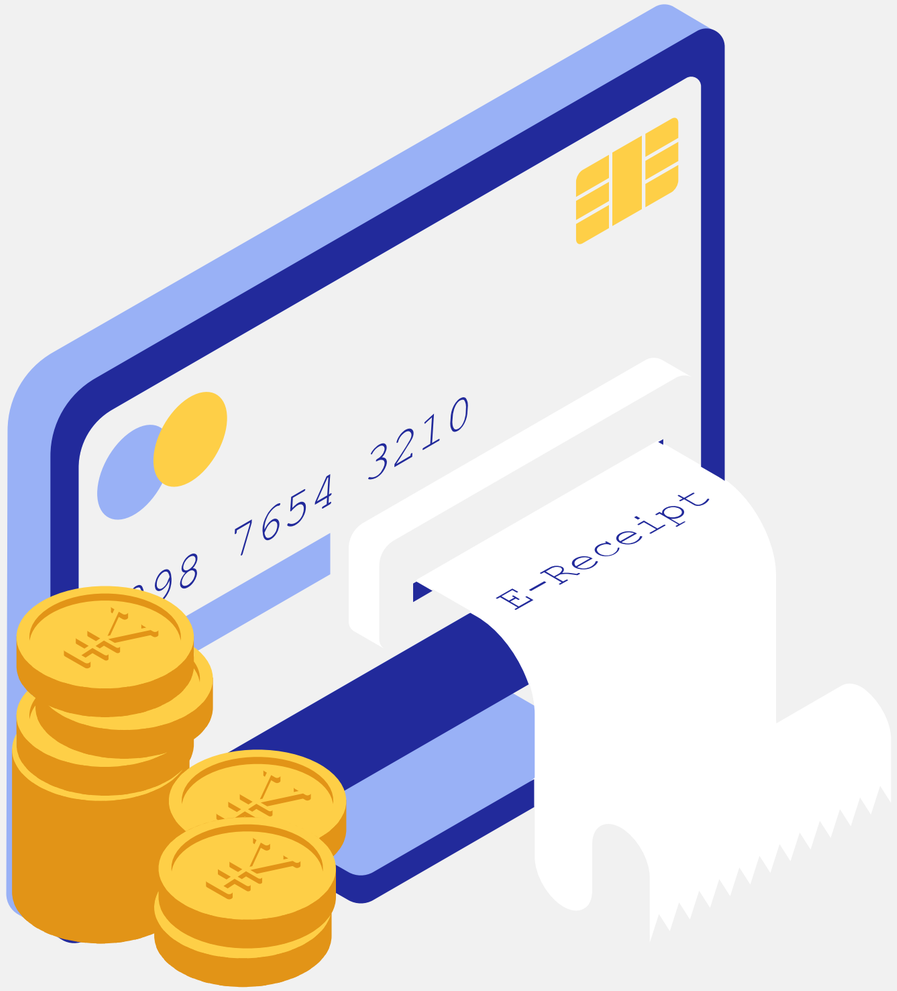


Data Mining Project 2026

E-commerce Behavior Analysis

Maha Gharsalli
Rihem Abdelmoumen
Rayen Ghourabi

Dataset Overview

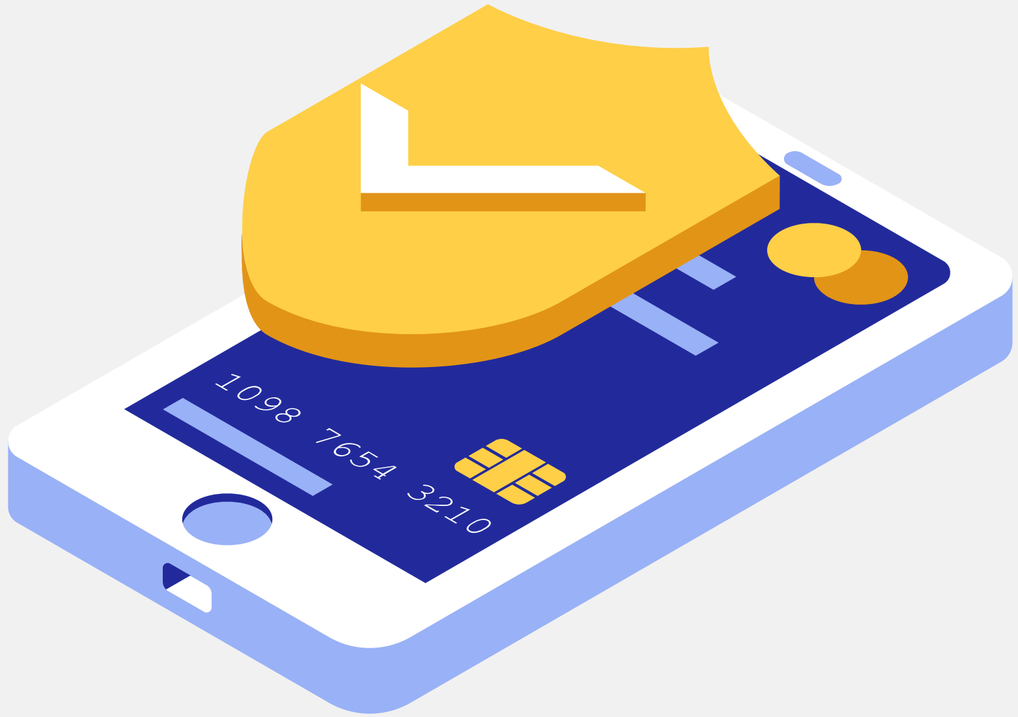
Dataset captures user behavior during online shopping sessions.

- Source: Kaggle
- ~8,000 user sessions
- 14 variables
- Type: Behavioral + demographic data
- Target: Purchase (0 / 1)
- Optional small examples:
- Age, Gender
- Time on site, Pages viewed
- Cart items, Previous purchases

Problem Statement

Challenge: converting user activity into purchases through better insights.

- Low conversion rate
- Users browse without purchasing
- Weak personalization strategies
- Ineffective marketing targeting
- Limited understanding of user behavior



Project Objectives

Goal: transform user data into actionable insights

- Predict user purchase behavior
- Understand and analyze customer behavior
- Identify key factors influencing decisions
- Segment customers based on behavior
- Support marketing strategy optimization

Data Preparation

Data Cleaning

- Purpose: Ensure dataset quality and remove useless or redundant information
- Removed duplicate rows
- → Avoids bias and repeated patterns in learning
- Dropped user_id column
- → Identifier has no predictive value
- → Prevents data leakage (model memorizing users instead of behavior)

```
# ____________________ Remove duplicates _____
df = df.drop_duplicates().reset_index(drop=True)
```

```
# ____________________ Drop identifier _______
df = df.drop(columns=['user_id'])
```

Missing Values & Outliers

- Purpose: Handle data quality issues without losing important information
- Missing values handled using:
- Median (numerical features) → robust to outliers
- Mode (categorical features) → keeps most common category
- No row deletion used
- → Preserves dataset size and rare cases
- Outliers detected using IQR method
- → Only 8 outliers found in session time
- → Kept because models (especially tree-based) are robust

```
# ____________________ Handle Missing Values ____________________

# Fill numeric values with median
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Fill categorical values with mode
cat_cols = df.select_dtypes(include='object').columns
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

Encoding Categorical Variables

- Purpose: Convert text data into numeric format for ML models
- Label EncodingGender → Female = 0, Male = 1
- One-Hot EncodingDevice Type → Desktop / Mobile / Tablet
- Result:
- → All categorical variables converted into numeric form
- → Dataset becomes fully machine-readable

```
# ____________________ Encoding ____________________
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])

df = pd.get_dummies(df, columns=['device_type'], prefix='device')

#_____________ Convert bool to int_____________________
bool_cols = df.select_dtypes(include='bool').columns
df[bool_cols] = df[bool_cols].astype(int)
```

Train-Test Split

- Purpose: Evaluate model performance on unseen data
- Split ratio: 80% training / 20% testing
- Stratified split applied
- → Keeps same class distribution in both sets
- Ensures fair evaluation of model performance
- Prevents bias caused by uneven class distribution

```
# ____________________ Train Test Split ____________________
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

SMOTE (Class Imbalance Handling)

- Purpose: Fix extremely imbalanced dataset
- Problem:
- → Very few non-buyers compared to buyers
- Solution: SMOTE (Synthetic Minority Oversampling Technique)Generates synthetic samples of minority class
- Based on nearest neighbors in feature space
- Result:
- → More balanced training data
- → Better learning for minority class patterns
- Applied only on training set
- → Prevents data leakage into test set

```
# ____________________ SMOTE (Partial Balancing) ____________________
smote = SMOTE(
    sampling_strategy=0.4,   # 40% balance (not equal)
    random_state=42
)

X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)

print("After SMOTE:")
print(y_train_bal.value_counts())
```

Feature Scaling

- Purpose: Normalize feature values for better model performance
- Used StandardScalerConverts data to mean = 0, standard deviation = 1
- Applied only on training data first
- → Then applied to test data
- Why important:
- Improves performance of distance-based models (KNN, Logistic Regression)
- Ensures all features contribute equally
- Binary features (0/1) were not scaled

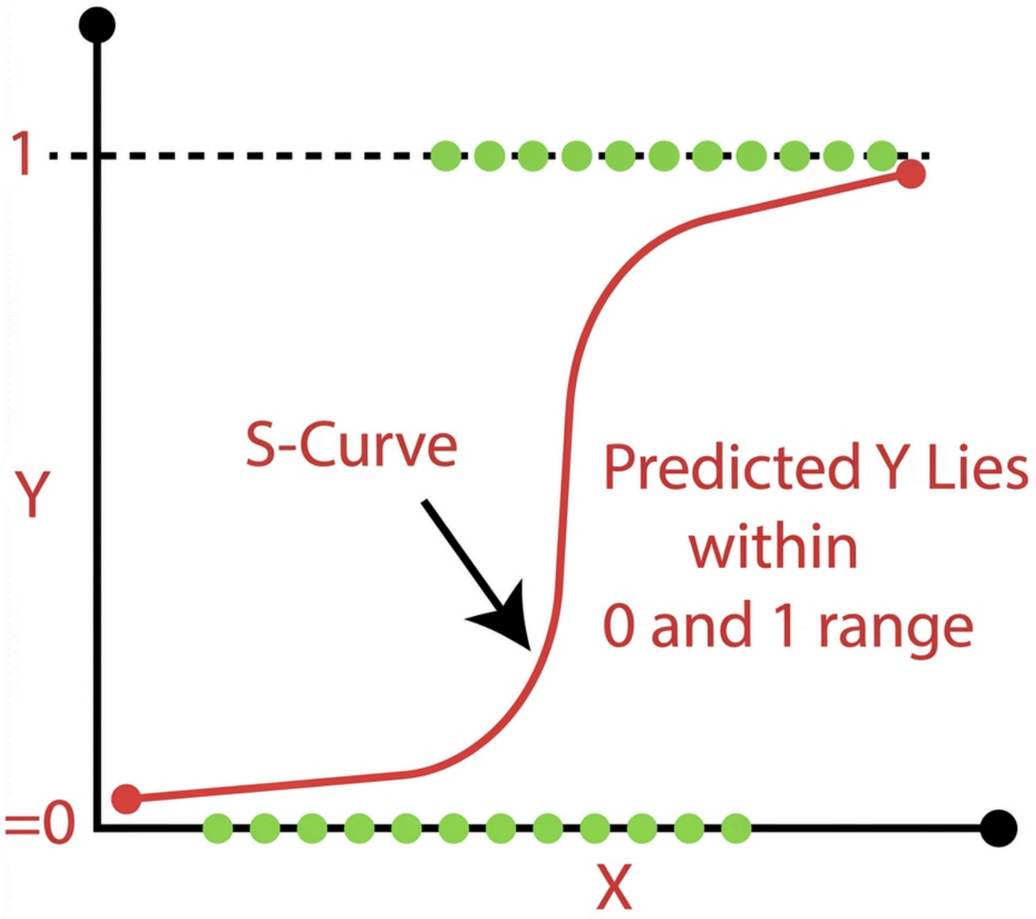
```
# ____________________ Feature Scaling ____________________
scale_cols = [
    'age', 'time_on_site', 'pages_viewed',
    'previous_purchases', 'cart_items',
    'avg_session_time', 'bounce_rate'
]

scaler = StandardScaler()

X_train_bal[scale_cols] = scaler.fit_transform(X_train_bal[scale_cols])
X_test[scale_cols] = scaler.transform(X_test[scale_cols])
```

Data Mining Methods & Analysis

Logistic Regression



Model Advantages:

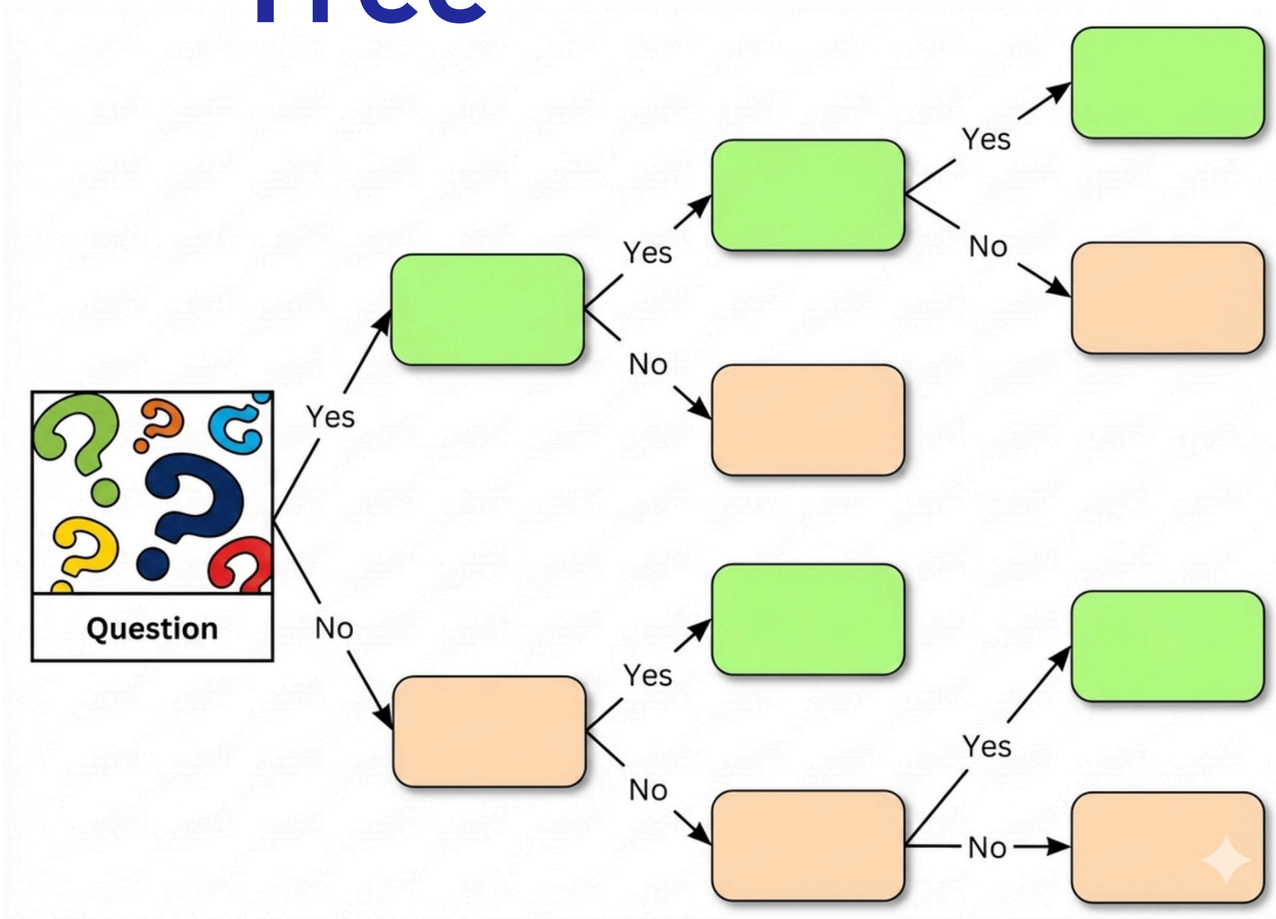
- Simple, reliable baseline.
- Understand which features drive purchase or non-purchase decisions.

Training & prediction

```
lr = LogisticRegression(class_weight=None, max_iter=1000, random_state=42)
lr.fit(X_train_bal, y_train_bal)
y_pred_lr = lr.predict(X_test)
```

Parameter	Purpose
lr.fit(X_train_bal, y_train_bal)	Trained on SMOTE-balanced training set to avoid leakage
y_pred_lr = lr.predict(X_test)	Applied to the test set for classification into “Buy” and “No Buy” using a probability threshold.

Decision Tree



Why Decision Tree?:

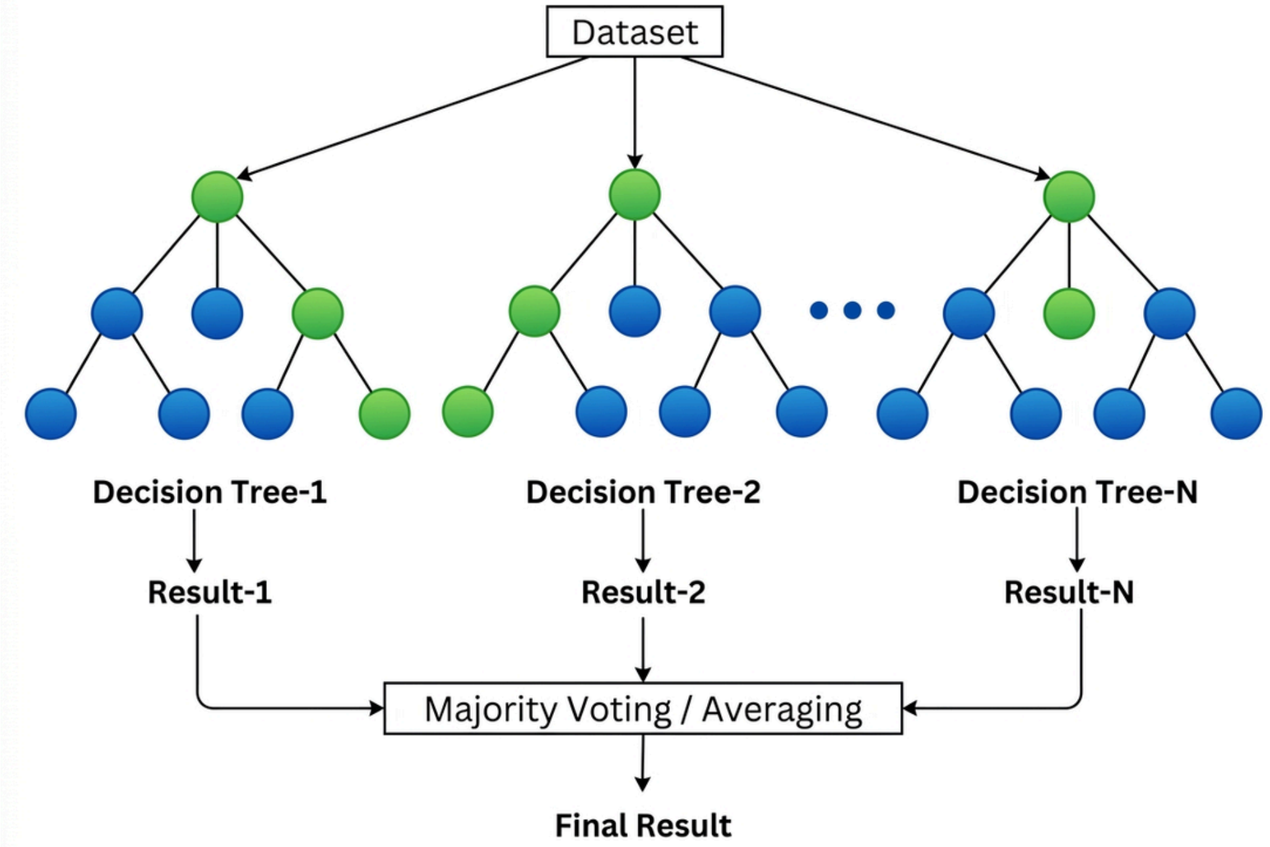
- Rule-based model
- Captures hierarchical decision structures
- Interpretable results for segmentation.

Implementation & Logic

```
dt = DecisionTreeClassifier(max_depth=5, class_weight='balanced', random_state=42)
dt.fit(X_train_bal, y_train_bal)
y_pred_dt = dt.predict(X_test)
```

Parameter	Purpose
max_depth=5	Cap tree at 5 levels to avoid overfitting and ensure generalizable splits.
dt.fit(X_train_bal, y_train_bal)	Learns by recursively splitting the data to reduce impurity.
y_pred_dt = dt.predict(X_test)	Predictions are made by routing each test sample through the tree until it reaches a leaf node.

Random Forest



Strategic Rationale:

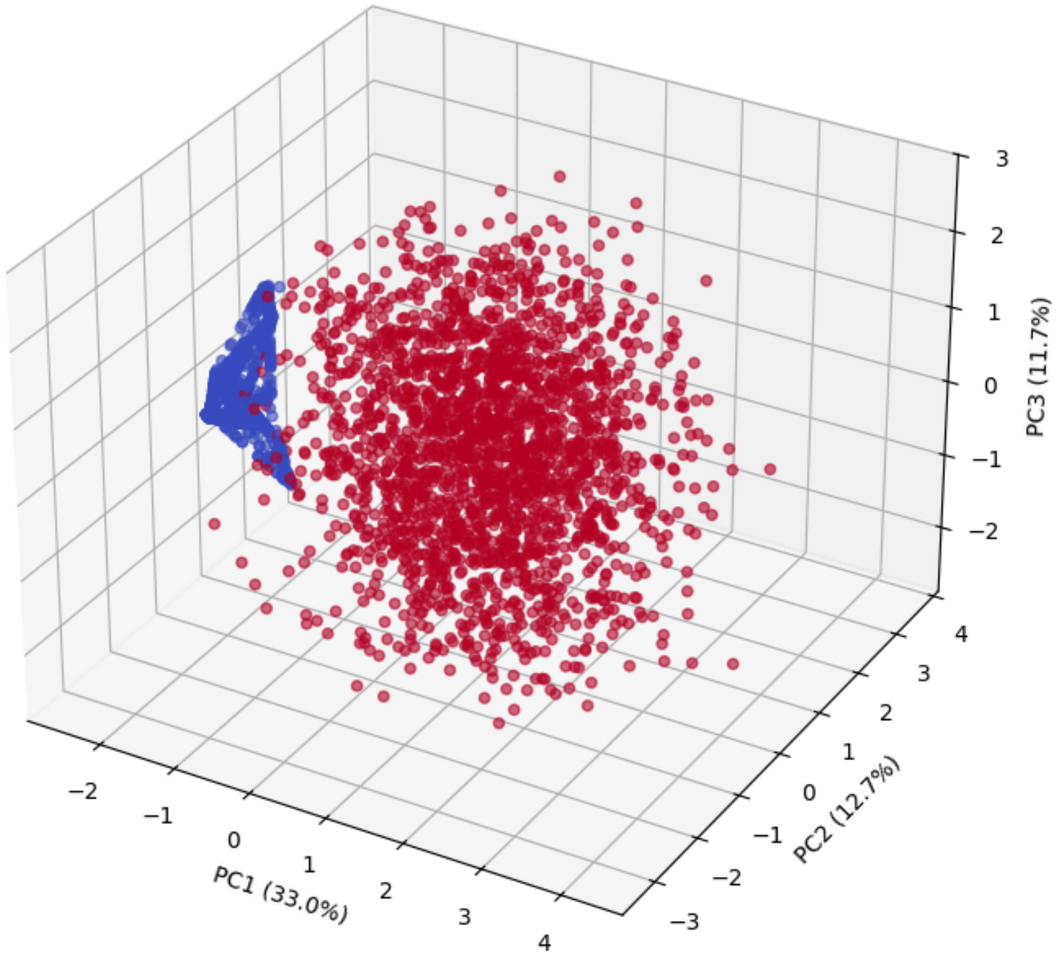
- Delivers enhanced predictive performance across complex data patterns.
- Provides robust, consistent insights.

Random Forest Execution & Voting

```
rf = RandomForestClassifier(
    n_estimators=200, max_depth=10, class_weight=None, random_state=42, n_jobs=-1)
rf.fit(X_train_bal, y_train_bal)
y_pred_rf = rf.predict(X_test)
```

Parameter	Purpose
n_estimators=200	Builds 200 trees to improve stability and reduce variance.
max_depth=10	Deeper trees limit overfitting
rf.fit(X_train_bal, y_train_bal)	Trains using random samples of the data and random subsets of features.
y_pred_rf = rf.predict(X_test)	the final class is determined by a majority vote.

3D PCA



Why 3D PCA?:

- Noise Reduction
- Highlights the spatial separation between target classes for sharper classification.
- Identifies the primary drivers that contribute most to model variation.

Evaluation

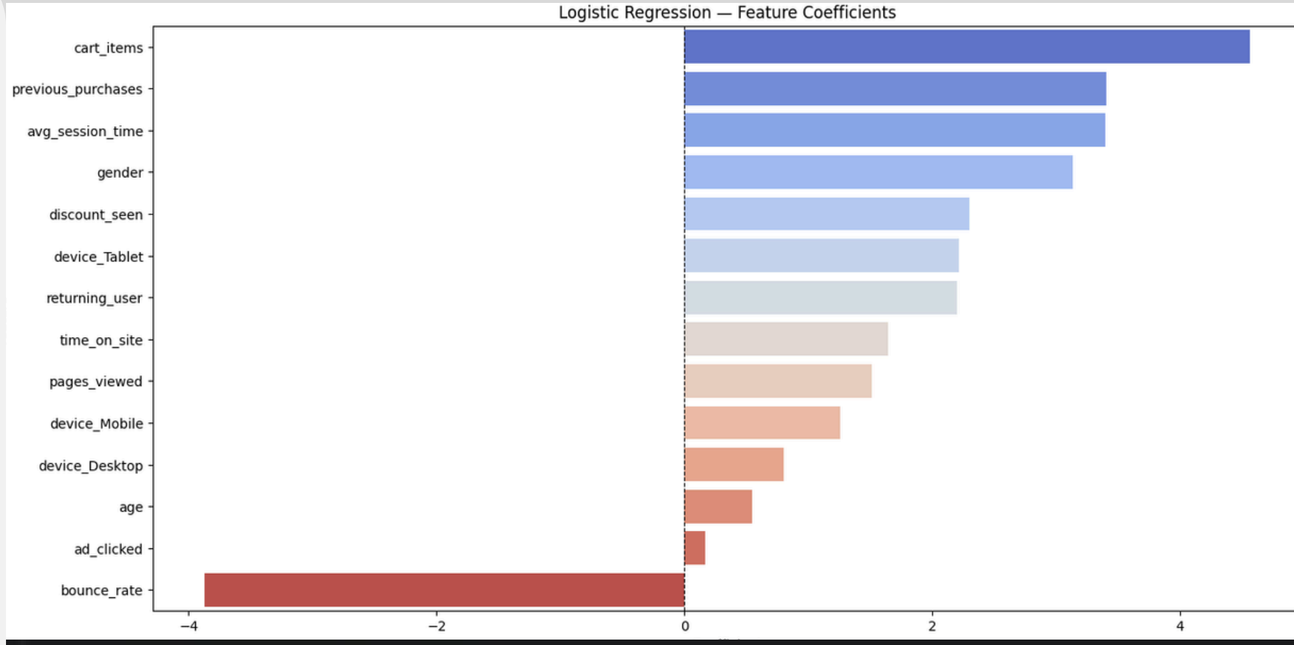
To evaluate model performance, we used four complementary metrics:

accuracy_score	weighted f1_score	roc_auc_score	classification_report
Gives the overall proportion of correct predictions.	Accounts for class imbalance by balancing precision and recall.	Measures the model's ability to distinguish between classes across all thresholds.	provides a detailed breakdown of performance per class, including macro and weighted averages.

Results And Interpretations

1. Logistic Regression – Best for identifying non-buyers

- Top drivers: cart_items, avg_session_time, previous_purchases (strong positive)
- Only strong negative: bounce_rate
- Performance on minority class (No Buy): Recall = 67% (best of all models)
- ROC-AUC = 99.85% → best-calibrated probabilities
- Use for: propensity scoring & churn detection



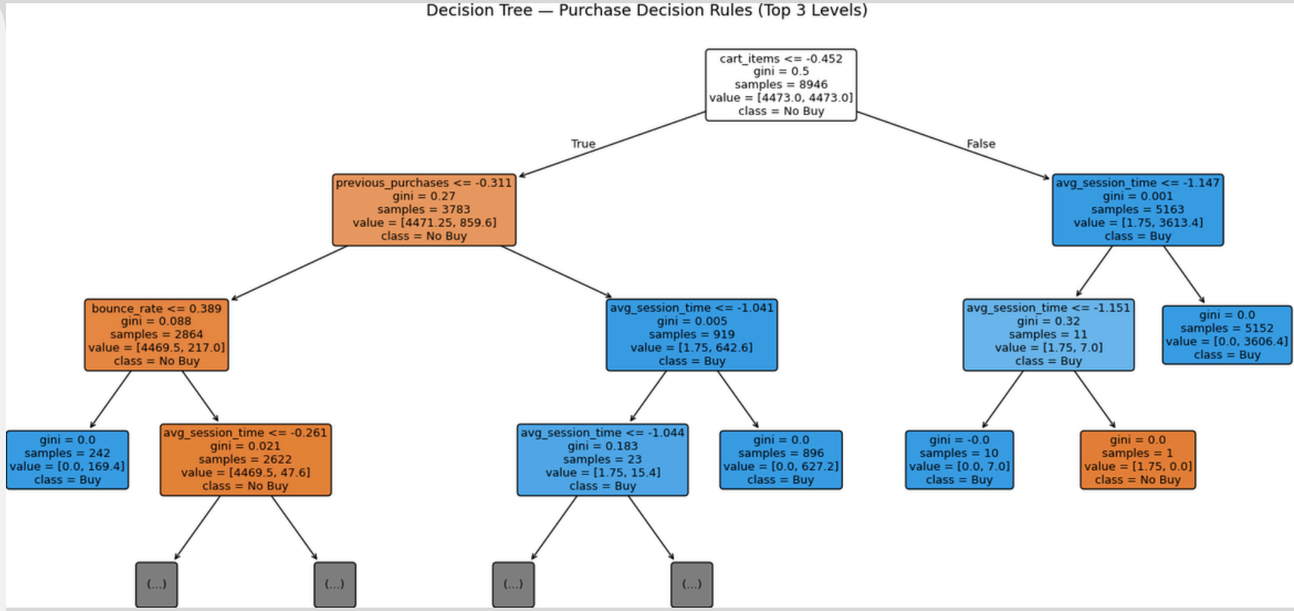

```
=== Logistic Regression ===
Accuracy : 0.9969
F1 Score : 0.9974
ROC-AUC : 0.9985
              precision    recall  f1-score   support

      No Buy       0.33      0.67      0.44         3
         Buy       1.00      1.00      1.00      1597

    accuracy                           1.00      1600
   macro avg       0.67      0.83      0.72      1600
weighted avg       1.00      1.00      1.00      1600
```

2. Decision Tree – Most interpretable

- Root split: cart_items → most important feature
- Pure buyer segment found: 5,152 users (high cart + high engagement)
- Minority recall = 33% (only catches 1 of 3 non-buyers)
- ROC-AUC = 83% → poor probability ranking
- Use for: extracting business rules & customer segmentation



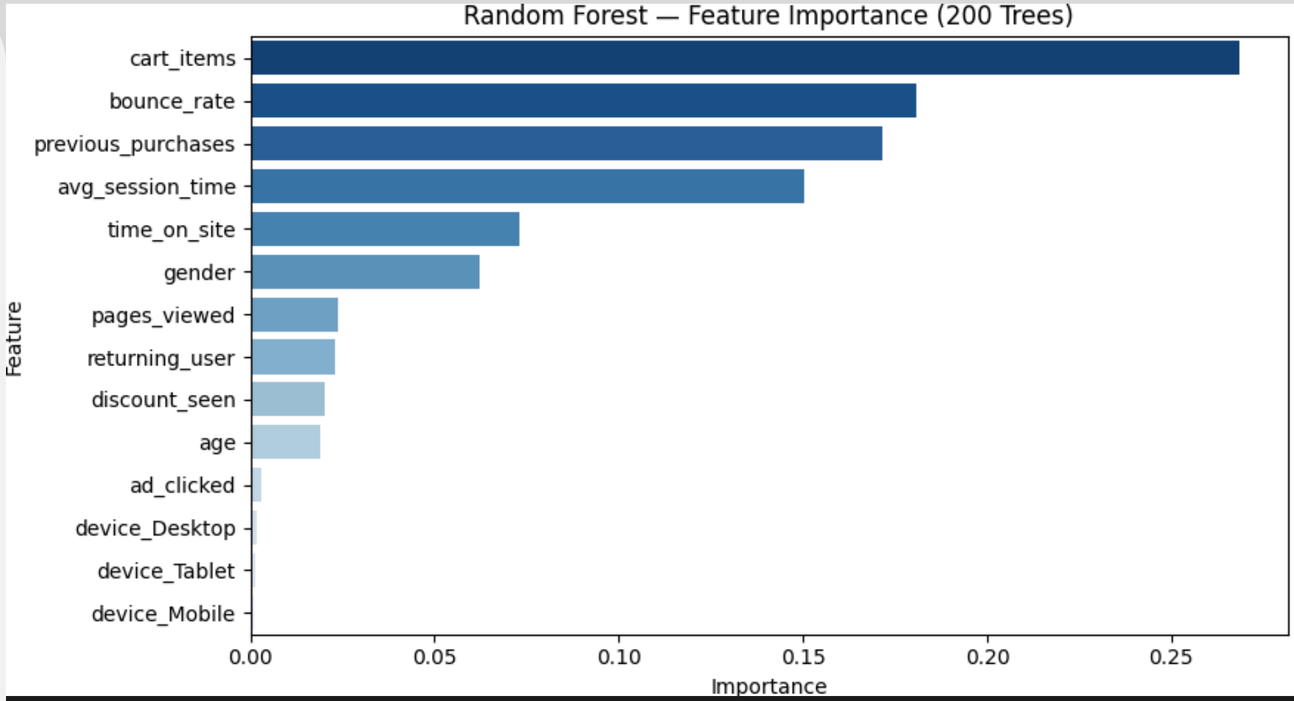

```
=== Decision Tree ===
Accuracy : 0.9981
F1 Score : 0.9979
ROC-AUC : 0.8323
              precision    recall  f1-score   support

      No Buy       0.50      0.33      0.40         3
         Buy       1.00      1.00      1.00      1597

    accuracy                           1.00      1600
   macro avg       0.75      0.67      0.70      1600
weighted avg       1.00      1.00      1.00      1600
```

3. Random Forest – Most stable feature ranking, but fails on minority

- Top features: cart_items, bounce_rate, previous_purchases
- ROC-AUC = 99.79% (excellent)
- Critical failure: 0% recall on "No Buy" → predicts every user as buyer
- Macro F1 = 0.50 (worst among all models)
- Not useful for identifying non-buyers




```
=== Random Forest ===
Accuracy : 0.9981
F1 Score : 0.9972
ROC-AUC : 0.9979
              precision    recall  f1-score   support

      No Buy       0.00      0.00      0.00         3
         Buy       1.00      1.00      1.00      1597

    accuracy                           1.00      1600
   macro avg       0.50      0.50      0.50      1600
weighted avg       1.00      1.00      1.00      1600

PS C:\Users\ramez\AppData\Local\Programs\Microsoft VS Code>
```

4. Confusion Matrix Summary (Key Trade-off)

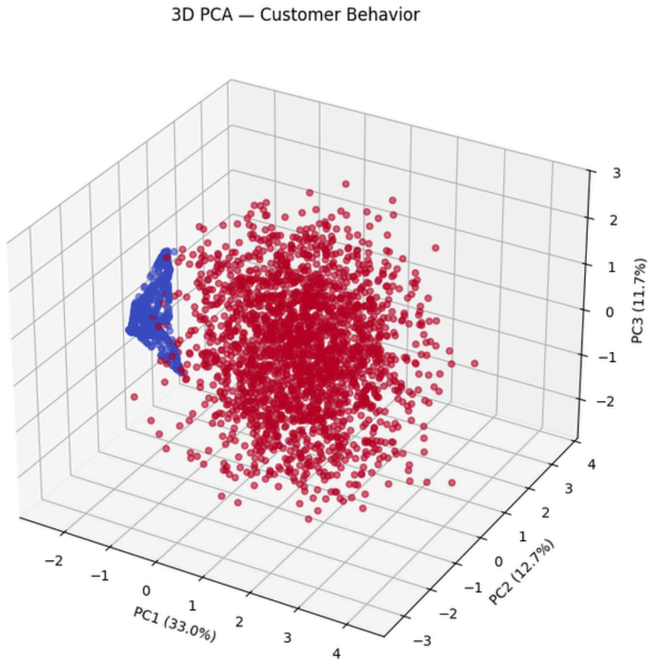
Model	No Buy Recall	No Buy Precisior	Total Errors
Logistic Regress	67%	33%	6
Decision Tree	33%	50%	3
Random Forest	0%	0%	3

5. PCA – 3D Validation

- PC1 (33% of variance) = "engagement intensity" axis
- Non-buyers cluster at low PC1 (low time, few pages, few cart items)
- Buyers spread across entire space → many behavioral paths to purchase

```
PC1: 0.3302
PC2: 0.1269
PC3: 0.1172
Total Explained: 0.5744

Top features influencing behavior (PC1):
time_on_site          0.494672
avg_session_time      0.489509
cart_items            0.375778
bounce_rate           0.373287
previous_purchases    0.371010
pages_viewed          0.228513
age                   0.137394
gender                0.123060
returning_user        0.067745
discount_seen         0.059813
Name: PC1, dtype: float64
```



6. Final Business Takeaways

- Engagement & cart activity drive purchases – not demographics
- Bounce rate is the strongest negative signal
- Discounts & ad clicks have weak impact
- High accuracy does not mean good minority-class performance
- Best model depends on goal:
  Find non-buyers → Logistic Regression
  Explain rules → Decision Tree
  Probability scores → Random Forest (but ignores minority class)

Conclusion

- **Goal Achieved:** Built a predictive framework to understand and model purchase behavior
- **Core Challenge:** Extreme class imbalance limited the ability to detect non-buyers despite high accuracy
- **Key Insight:**
- → User behavior (engagement & cart activity) is the true driver of purchases
- → Demographics and discounts have limited influence
- **Business Value:**
- → Focus on improving user experience and interaction
- → Apply targeted strategies for diverse non-buyer segments
- **Final Message:**
- → Prediction alone is not enough — understanding behavior creates real impact
- → Data-driven insights are essential for optimizing conversion strategies

THANK YOU